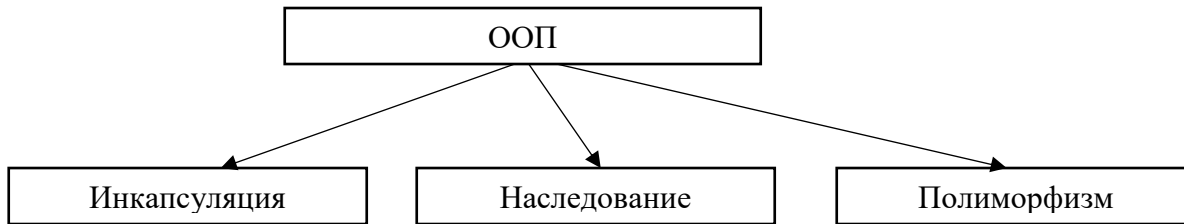


Полиморфизм

1. Концепции (принципы) ООП



Инкапсуляция – заключение переменных и функций, относящихся к одной сущности, в единый объект («капсулу») и регулирование доступа к ним со стороны других сущностей. Иногда заключение переменных и функций в единый объект выделяют в отдельную концепцию – **абстракцию** (**абстрагирование**).

Наследование – механизм, позволяющий определить новые классы на базе уже существующих («наследуя» их поля и методы и расширяя их функционал).

Полиморфизм – механизм, позволяющий создавать различные реализации одноимённых методов и вызывать «подходящую» версию в зависимости от типов параметров и/или объекта.

2. Виды полиморфизма

В теме «Наследование» вы уже познакомились с переопределением методов (введение нового определения в классе-наследнике), и вызывали их в объектах-наследниках. Таким образом, вы создавали методы с одинаковыми названиями, но разным функционалом. И это подходит под определение полиморфизма 😊

Переопределение методов в классах-наследниках и уже знакомый вам механизм шаблонов – это проявление **статического полиморфизма**. т.к. то, какой метод будет вызван для объекта, известно на этапе компиляции.

Если есть статический полиморфизм, то должен быть и динамический? И он есть 😊 **Динамический полиморфизм** проявляется тогда, когда конкретный метод определяется уже во время работы программы.

С полиморфизмом тесно связано понятие **связывания** – сопоставление вызова функции с её определением. В зависимости от типа полиморфизма выделяют соответственно раннее (статическое) и позднее (динамическое) связывание.

Примеры полиморфизма приведены в примере 1 с классами прямоугольника и прямоугольного параллелепипеда. В строке 44 срабатывает статический полиморфизм: вызывается переопределённый метод класса-наследника. В строках 45 и 47 создаются указатель и ссылка типа **базового класса**, в которые помещаются адрес и объект **производного класса** соответственно. В строках 46 и 48 для них вызываются методы для подсчёта площади, и срабатывает метод родительского класса.

Пример 1. Полиморфизм

```
1  #include <iostream>
2
3  class Rectangle
4  {
5  protected:
6      float length;
7      float width;
8  public:
9      Rectangle(const float &l = 0, const float &w = 0) : length(l), width(w) {}
10     float countPerimeter() const
11     {
12         return 2 * (length + width);
13     }
14     float countArea() const
15     {
16         return length * width;
17     }
18 };
19
20
21 class Parallelepiped : public Rectangle
22 {
23     float height;
24 public:
25     Parallelepiped(const float &l = 0, const float &w = 0, const float &h = 0)
26         : Rectangle(l, w), height(h) {}
27     float countPerimeter() const
28     {
29         return 4 * (length + width + height);
30     }
31     float countVolume() const
32     {
33         return length * width * height;
34     }
35     float countArea() const
36     {
37         return 2 * (length * width + length * height + width * height);
38     }
39 };
40
41 int main()
42 {
43     Parallelepiped p(1, 2, 3);
44     std::cout << "p's area: " << p.countArea() << std::endl;
45     Rectangle* r = &p;
46     std::cout << "r's area: " << r->countArea() << std::endl;
47     Rectangle& r2 = p;
48     std::cout << "r2's area: " << r2.countArea() << std::endl;
49     return 0;
50 }
```

Результат будет таким.

```
p's area: 22
r's area: 2
r2's area: 2
```

Так, а динамический полиморфизм-то где?

Чтобы сделать полиморфизм динамическим и получить возможность вызывать методы в соответствии с классом объекта, а не ссылки/указателя, необходимо в родительском классе сделать потенциально переопределяемые методы виртуальными, указав модификатор `virtual` в начале объявления метода (пример 2).

```
p's area: 22
r's area: 22
r2's area: 22
```

Пример 2. Класс с виртуальными функциями

```
1 class Rectangle
2 {
3 protected:
4     float length;
5     float width;
6 public:
7     Rectangle(const float &l = 0, const float &w = 0) : length(l), width(w) {}
8     virtual float countPerimeter() const
9     {
10         return 2 * (length + width);
11     }
12     virtual float countArea() const
13     {
14         return length * width;
15     }
16 };
```

Стоит отметить, что если при переопределении функции не объявить виртуальными, они таковыми всё равно будут являться (т.к. в родительском классе были виртуальными).

Для работы динамического полиморфизма кол-во параметров, их типы, а также тип возвращаемого значения для методов должны полностью совпадать. Переопределением константного метода может являться только константный метод, неконстантного – неконстантный. Из этого правила есть только одно исключение – когда типом возвращаемого значения является указатель или ссылка на текущий класс.

Чтобы не иметь проблем, связанных с различиями в параметрах методов (типах параметров, константности метода), **настоятельно рекомендуется** в конце объявления переопределённой функции добавлять ключевое слово `override` (пример 3). В случае несовпадения прототипов методов код не скомпилируется.

Пример 3. Переопределение с ключевым словом `override`

```
1 class Parallelepiped : public Rectangle
2 {
3     float height;
4 public:
5     Parallelepiped(const float &l = 0, const float &w = 0, const float &h = 0)
6         : Rectangle(l, w), height(h) {}
7     float countPerimeter() const override;
8     float countVolume() const;
9     float countArea() const override;
10    ~Parallelepiped() {}
11 };
```

В теме «Наследование» упоминался спецификатор `final`, который запрещал наследоваться от класса. Это слово можно использоваться для методов, чтобы запретить их переопределять (пример 4).

Пример 4. Переопределение с ключевым словом `final`

```
1 class Parallelepiped : public Rectangle
2 {
3     float height;
4 public:
5     Parallelepiped(const float &l = 0, const float &w = 0, const float &h = 0)
6         : Rectangle(l, w), height(h) {}
7     float countPerimeter() const override final;
8     float countVolume() const;
9     float countArea() const override final;
10    ~Parallelepiped() {}
11 };
```

3. Как это работает?

При компиляции для каждого класса, содержащего виртуальные методы или наследованного от класса, имеющего виртуальные методы, создаётся скрытый член – **таблица виртуальных методов**, в которой для каждого виртуального метода содержится адрес памяти, по которому находится подходящая реализация метода. Если метод переопределён в наследнике, в его таблице виртуальных методов будет указан адрес переопределения, а если не был переопределён, то адрес метода из класса-родителя.

4. Полиморфизм при создании и уничтожении объектов

При вызовах виртуальных функций в конструкторах и деструкторах всегда будет вызываться метод из родительского класса. Это связано с тем, что создание объекта начинается с родительской части, а уничтожение – с части наследника, поэтому к моменту вызова метода в теле конструктора часть от наследника ещё не будет создана, а в теле деструктора – уже будет уничтожена.

+1 к best practices: не вызывайте виртуальные методы в конструкторах и деструкторах.

5. Виртуальные деструкторы

С учётом новых знаний о работе с объектами-наследниками через ссылки и указатели родительского класса может возникнуть вопрос: раз мы можем объявить указатель типа родительского класса на объект производного класса, то **что будет происходить, если попытаться освободить память через указатель родительского класса?**

Исходя из опыта с обычными функциями, на этот вопрос можно ответить без подвоха: **если деструктор не объявлен как виртуальный, будет вызван деструктор по типу указателя, т.е. деструктор родительского класса.** Если объект имеет собственные динамические поля, произойдёт утечка памяти. Выход из этой ситуации – объявления деструктора виртуальным (а вот конструкторы виртуальными быть не могут).

+1 к best practices: если планируете наследоваться от класса, объявляйте его деструктор виртуальным.

6. А зачем это нужно?

Это главный вопрос, который должен был остаться после прочтения вышеизложенного. Зачем объявлять указатель или ссылку родительского типа на объект типа наследника?

Как вы помните, есть достаточно много структур данных, позволяющих работать с множеством объектов (статический массив, динамический массив, вектор, очередь, список, множество и т.д.), но все они работают с объектами одного типа.

Динамический полиморфизм позволяет в рамках одной структуры-множества работать с объектами разных типов, объединёнными общим предком, причём делать это единообразно (через одни и те же функции). Например, как в примере 5: в первом цикле накидали в вектор прямоугольников и параллелепипедов, а во втором вывели на экран параметры, периметр и площадь для всех фигур, не задумываясь о том, к объекту какого типа идёт обращение. Методы печати, само собой, должны быть предварительно объявлены в классах, причём как минимум метод `Rectangle` должен быть объявлен виртуальным.

Пример 5. Единообразная работа с объектами разных типов

```
1 void Rectangle::printParameters() const
2 {
3     std::cout << "(" << length << ", " << width << ") \n";
4 }
5
6 void Parallelepiped::printParameters() const
7 {
8     std::cout << "(" << length << ", " << width << ", " << height << ") \n";
9 }
10
11 int main()
12 {
13     std::vector<Rectangle*> figures;
14     for (size_t i = 0; i < 4; ++i)
15         if (i % 2)
16             figures.push_back(new Parallelepiped(i, i + 1, i + 2));
17         else
18             figures.push_back(new Rectangle(i + 1, i + 2));
19
20     for (Rectangle* const& figure : figures)
21     {
22         figure->printParameters(); // Печатаем параметры фигуры
23         // Выводим периметр и площадь
24         std::cout << "perimeter: " << figure->countPerimeter() << " \n";
25         std::cout << "area: " << figure->countArea() << " \n";
26     }
27     return 0;
28 }
```

```
(1, 2)
perimeter: 6
area: 2
(1, 2, 3)
perimeter: 24
area: 22
(3, 4)
perimeter: 14
area: 12
(3, 4, 5)
perimeter: 48
area: 94
```

7. Чистые виртуальные функции и абстрактные классы

Есть особая разновидность виртуальных функций, которые не имеют определения в родительском классе и должны быть определены в классе-наследнике. Такие функции называются чистыми виртуальными функциями. В примере 6 методы `countPerimeter` и `countArea` класса `Figure` являются чистыми виртуальными функциями: им «присвоено» значение 0.

Пример 6. Чистые виртуальные функции

```
1  class Figure
2  {
3  public:
4      virtual float countPerimeter() const = 0;
5      virtual float countArea() const = 0;
6  };
7
8  class Rectangle : public Figure
9  {
10 protected:
11     float length;
12     float width;
13 public:
14     Rectangle(const float &l = 0, const float &w = 0) : length(l), width(w) {}
15     float countPerimeter() const override
16     {
17         return 2 * length * width;
18     }
19     float countArea() const override
20     {
21         return length * width;
22     }
23 };
```

Т.к. функция не определена, её нельзя использовать. Следствие этого – нельзя создать объект класса, который имеет хотя бы одну чистую виртуальную функцию. Такие классы называются абстрактными классами. Если в наследнике абстрактного класса чистая виртуальная функция не определена, этот класс тоже будет абстрактным.

Однако вне класса чистую виртуальную функцию можно определить, как показано в примере 7. Непосредственно в теле класса функция так и остаётся с «присвоением» 0, класс остаётся абстрактным. В этом случае классы-наследники до переопределения методов будут являться абстрактными классами, но смогут использовать «реализацию по умолчанию», явно вызывая её через имя родительского класса (`Figure::countPerimeter()`).

Абстрактные классы нужны для задания некоторого «шаблона». Сами по себе, в отрыве от наследников, они могут не иметь смысла. Например, фигура без параметров не имеет смысла, но отсутствие полей вроде «сторона» в базовом классе позволяет определять более разных наследников (например, круг не имеет сторон, но имеет периметр и площадь, объявленные в родительском классе). Несмотря на то, что нельзя создать объект фигуры, этот тип данных можно использовать для работы с объектами-наследниками (указатели и ссылки не порождают новые объекты, а значит могут иметь тип абстрактного класса).

Пример 7. Определение чистых виртуальных функций

```
1 float Figure::countPerimeter() const
2 {
3     return 0;
4 }
5
6 float Figure::countArea() const
7 {
8     return 0;
9 }
10
11 float Rectangle::countPerimeter() const
12 {
13     return Figure::countPerimeter();
14 }
15
16 float Rectangle::countArea() const
17 {
18     return Figure::countArea();
19 }
```

8. Интерфейс

Интерфейс – это класс, не имеющий полей, все методы которого являются чистыми виртуальными функциями.

В C++ нет специального ключевого слова для объявления интерфейсов (как, например, в C#), они определяются как обычные абстрактные классы. Например, класс `Figure` из примера 6 является интерфейсом.

9. Доступ к полям наследника через родительский класс

У динамического полиморфизма есть неприятная сторона: указатель/ссылка родительского типа «видит» только свои поля и методы. Таким образом, становится затруднительно вызвать те методы, которые есть только в классе-наследнике.

Из этой ситуации есть 2 выхода:

- 1) добавить соответствующую виртуальную функцию в базовый класс;
- 2) использовать преобразование (приведение) указателя или ссылки к типу производного типа (указателю или ссылке).

Первый подход плох для узкоспециализированных функций наследника, поскольку производные классы зависят от базового, но базовый не должен зависеть от производных. Например, довольно странно в класс фигуры будет добавлять метод для вычисления диаметра, поскольку такие характеристики, как радиус и диаметр есть только у специфичных типов фигур. В подобных случаях стоит использовать второй подход.

Преобразование типа также имеет свои ограничения: оно не преобразует классы, не имеющие виртуальных таблиц (т.е. ни одного виртуального метода), и классы, использующие виртуальное или `private`- или `protected`-наследование.

Преобразование типов обеспечивает `dynamic_cast`. Если преобразование типа не удалось, `dynamic_cast` возвращает `nullptr`.

Такое преобразование типа – родителя к наследнику – называется **понижающим преобразованием (приведением) типа**. Обратный процесс, когда наследник преобразуется к типу родителя, называется **повышающим приведением типа**, которое может быть осуществлено с помощью `static_cast`.

Пример 8. Понижающее преобразование типа

```

1  int main()
2  {
3      std::vector<Rectangle*> figures;
4      for (size_t i = 0; i < 4; ++i)
5          if (i % 2)
6              figures.push_back(new Parallelepiped(i, i + 1, i + 2));
7          else
8              figures.push_back(new Rectangle(i + 1, i + 2));
9
10     for (Rectangle* const& figure : figures)
11     {
12         figure->printParameters(); // Выводим параметры фигуры
13         // Попытка выполнить преобразование типа и вывод объёма в случае успеха
14         if (Parallelepiped* p = dynamic_cast<Parallelepiped*>(figure))
15             std::cout << "volume: " << p->countVolume() << "\n";
16         // Выводим периметр и площадь
17         std::cout << "perimeter: " << figure->countPerimeter() << "\n";
18         std::cout << "area: " << figure->countArea() << "\n";
19     }
20     return 0;
21 }

```

```

(1, 2)
perimeter: 6
area: 2
(1, 2, 3)
volume: 6
perimeter: 24
area: 22
(3, 4)
perimeter: 14
area: 12
(3, 4, 5)
volume: 60
perimeter: 48
area: 94

```

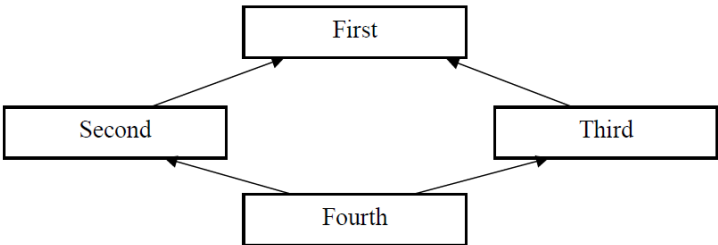
10. Виртуальное наследование

Как вы помните, при наследовании от двух и более классов, имеющих общего предка, возникало дублирование полей, наследованных от предка, которые в т.ч. могли содержать разные значения («Алмаз смерти»).

```

First constructor
Second constructor
First constructor
Third constructor
Fourth constructor
First: a = 0
Second: b = 1
First: a = 2
Third: c = 3
Fourth: d = 4
Fourth destructor
Third destructor
First destructor
Second destructor
First destructor

```



Для избавления от такого дублирования сущностей используется виртуальное наследование. Пример приведён в примере 9. Отличительной чертой являются спецификаторы `virtual` в объявлениях классов `Second` и `Third` (строки 23 и 43). Такая реализация позволяет создавать часть, наследованную от `First`, только 1 раз, причём за её создание теперь отвечает `Fourth`. Это позволяет `Fourth` вызывать конструктор `First` в своём конструкторе для инициализации наследованных полей.

Пример 9. Виртуальное наследование

```
1  #include <iostream>
2
3  class First
4  {
5      int a;
6  public:
7      First(const int &v = 0) : a(v)
8      {
9          std::cout << "First constructor\n";
10     }
11
12     void print()
13     {
14         std::cout << "First: a = " << a << std::endl;
15     }
16
17     ~First()
18     {
19         std::cout << "First destructor\n";
20     }
21 };
22
23 class Second : virtual public First
24 {
25     int b;
26 public:
27     Second(const int &v1 = 0, const int &v2 = 1) : First(v1), b(v2)
28     {
29         std::cout << "Second constructor\n";
30     };
31
32     void print()
33     {
34         First::print();
35         std::cout << "Second: b = " << b << std::endl;
36     }
37     ~Second()
38     {
39         std::cout << "Second destructor\n";
40     }
41 };
42
43 class Third : virtual public First
44 {
45     int c;
46 public:
47     Third(const int &v1 = 0, const int &v2 = 1) : First(v1), c(v2)
48     {
49         std::cout << "Third constructor\n";
50     }
51 }
```

Продолжение примера 9

```
52     void print()
53     {
54         First::print();
55         std::cout << "Third: c = " << c << std::endl;
56     }
57
58     ~Third()
59     {
60         std::cout << "Third destructor\n";
61     }
62 };
63
64 class Fourth : public Second, public Third
65 {
66     int d;
67 public:
68     Fourth(const int &v1 = 0, const int &v2 = 1,
69           const int &v3 = 2, const int &v4 = 3, const int &v5 = 4)
70         : Second(v1, v2), Third(v3, v4), d(v5)
71     {
72         std::cout << "Fourth constructor\n";
73     }
74
75     void print()
76     {
77         Second::print();
78         Third::print();
79         std::cout << "Fourth: d = " << d
80                 << std::endl;
81     }
82
83     ~Fourth()
84     {
85         std::cout << "Fourth destructor\n";
86     }
87 };
88
89 int main()
90 {
91     Fourth ex1;
92     ex1.print();
93     return 0;
94 }
```

```
First constructor
Second constructor
Third constructor
Fourth constructor
First: a = 0
Second: b = 1
First: a = 0
Third: c = 3
Fourth: d = 4
Fourth destructor
Third destructor
Second destructor
First destructor
```